

DAY 15 — PostgreSQL Fundamentals

You're now moving from **SQL querying** → **PostgreSQL engineering**.

Your SQL fundamentals are already strong, so I'll skip beginner material. Today we focus on how PostgreSQL behaves as a **production database**, especially for Data Engineering and RAG systems.

I'll also deliberately reinforce your weak area: **LEFT JOIN + filtering + NULL semantics**.

1. PostgreSQL vs SQL

First, separate these concepts:

SQL

SQL is a **language**.

Examples:

```
SELECT *  
FROM users;  
SELECT *  
FROM documents  
WHERE page_count > 50;
```

PostgreSQL

PostgreSQL is a **database management system** that implements SQL and provides additional capabilities.

PostgreSQL gives you:

- SQL
- transactions
- indexes
- constraints
- views
- CTEs
- window functions
- JSON/JSONB
- arrays
- full-text search
- extensions
- advanced data types
- query planner
- EXPLAIN
- concurrency control

Think:

SQL



Language

PostgreSQL



Database system implementing SQL

+ many additional capabilities

2. PostgreSQL Architecture — Mental Model

As a Data Engineer, understand this flow:

Application



| SQL



PostgreSQL Server



|— Parser



|— Analyzer



|— Planner / Optimizer



|— Executor



|— Storage

For example:

```
SELECT *
```

```
FROM documents
```

```
WHERE page_count > 50;
```

PostgreSQL roughly needs to:

1. Parse SQL



2. Understand tables/columns



3. Generate possible execution plans

↓

4. Choose a plan

↓

5. Execute plan

↓

6. Return rows

This becomes extremely important later when we study:

INDEXES

EXPLAIN

QUERY OPTIMIZATION

3. Database vs Schema vs Table

This distinction is important in PostgreSQL.

Think:

PostgreSQL Server

|

|— Database A

| |

| |— Schema

| | |— Table

| | └─ Table

| |

| └─ Schema

|

└─ Database B

A common PostgreSQL structure:

rag_platform

|

|— public

| |— users

| |— documents

| |— chunks

| └─ embeddings

```
|
└─ analytics
└─ document_metrics
└─ user_metrics
```

You can reference a table using:

schema.table

Example:

```
SELECT *
FROM public.documents;
```

4. PostgreSQL Data Types

PostgreSQL has many types.

For your Data Engineering work, know these particularly well.

Numeric

INTEGER

BIGINT

NUMERIC

DECIMAL

REAL

DOUBLE PRECISION

Example:

page_count INTEGER

For financial precision:

price NUMERIC(10,2)

5. Text

PostgreSQL supports:

TEXT

VARCHAR(n)

CHAR(n)

For most application/data-engineering use cases:

name TEXT

is perfectly reasonable.

Example:

```
CREATE TABLE users (
id BIGINT,
```

name TEXT

);

6. Boolean

BOOLEAN

Values:

TRUE

FALSE

NULL

Example:

is_processed BOOLEAN

Query:

```
SELECT *  
FROM documents  
WHERE is_processed = TRUE;
```

You can also write:

```
WHERE is_processed;
```

7. Date and Time

Important PostgreSQL types:

DATE

TIME

TIMESTAMP

TIMESTAMPTZ

INTERVAL

For production systems, pay particular attention to:

TIMESTAMPTZ

which represents a timestamp with time-zone awareness.

Example:

created_at TIMESTAMPTZ

Typical RAG table:

```
CREATE TABLE documents (  
id BIGINT,  
user_id BIGINT,  
title TEXT,  
created_at TIMESTAMPTZ  
);
```

8. UUID

PostgreSQL supports:

UUID

Example:

```
CREATE TABLE documents (  
id UUID,  
title TEXT  
);
```

UUIDs are commonly used for distributed systems.

For example:

550e8400-e29b-41d4-a716-446655440000

You will encounter UUIDs frequently in production APIs.

9. JSONB — VERY IMPORTANT FOR GENAI

For your GenAI/RAG career, this matters.

PostgreSQL supports:

JSON

JSONB

JSONB is a binary representation optimized for processing/querying JSON data.

Example:

```
CREATE TABLE documents (  
id BIGINT,  
title TEXT,  
metadata JSONB  
);
```

Insert:

```
INSERT INTO documents (  
id,  
title,  
metadata  
)  
VALUES (  
101,  
'RAG Fundamentals',  
'{"language": "en", "source": "pdf", "pages": 20}'  
);
```

Query:

```
SELECT *  
FROM documents  
WHERE metadata->>'language' = 'en';
```

Another example:

```
SELECT  
metadata->>'source' AS source  
FROM documents;
```

You'll eventually use PostgreSQL + JSONB for things such as:

document metadata

chunk metadata

LLM metadata

retrieval metadata

model configuration

API payloads

evaluation results

10. Arrays

PostgreSQL supports arrays.

Example:

```
CREATE TABLE documents (  
id BIGINT,  
title TEXT,  
tags TEXT[]  
);
```

Insert:

```
INSERT INTO documents (  
id,  
title,  
tags  
)  
VALUES (  
101,  
'RAG Fundamentals',  
ARRAY['rag', 'llm', 'embeddings']  
);
```

Query:

```
SELECT *
```

```
FROM documents
WHERE 'rag' = ANY(tags);
```

11. NULL — PostgreSQL Perspective

This is directly related to your Day 14 weakness.

Remember:

NULL $\neq$ 0

NULL $\neq$ ""

NULL $\neq$ FALSE

It means:

unknown / missing / absent value

Suppose:

users

id | name

---+-----

1 | Aarav

2 | Emma

3 | Liam

and:

documents

id | user_id

---+-----

101 | 1

102 | 1

103 | 2

Now:

```
SELECT
```

```
u.id,
```

```
u.name,
```

```
d.id AS document_id
```

```
FROM users u
```

```
LEFT JOIN documents d
```

```
ON u.id = d.user_id;
```

Conceptually:

user_id | name | document_id

-----+-----+-----

1 | Aarav | 101
1 | Aarav | 102
2 | Emma | 103
3 | Liam | NULL

That final row is crucial.

PostgreSQL creates a **NULL-extended row** for Liam because there was no matching document.

12. Your Weakness: ON vs WHERE

This is one of the most important concepts of Day 15.

Suppose we want:

All users, plus their processed documents.

Query A

```
SELECT
u.id,
u.name,
d.id AS document_id
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
AND d.status = 'processed';
```

The filtering condition is part of the **JOIN condition**.

So users without processed documents remain.

Conceptually:

Aarav → processed document

Emma → processed document

Liam → NULL

Now:

Query B

```
SELECT
u.id,
u.name,
d.id AS document_id
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
WHERE d.status = 'processed';
```

What happens?

For Liam:

`d.status = NULL`

Then:

`NULL = 'processed'`

doesn't evaluate to TRUE.

So the row is removed by WHERE.

Effectively:

`LEFT JOIN`

↓

WHERE condition on right table

↓

many unmatched rows disappear

↓

INNER JOIN-like behavior

This distinction must become automatic for you.

13. The Golden Rule

When using LEFT JOIN:

Filter the right table in ON

if you want to:

preserve unmatched left-side rows.

Example:

`LEFT JOIN documents d`

`ON u.id = d.user_id`

`AND d.status = 'processed'`

Filter in WHERE

if you intentionally want:

only rows having a matching condition.

Example:

`WHERE d.status = 'processed'`

This is not merely syntax.

It changes the **meaning of your dataset**.

14. COUNT(*) vs COUNT(column)

Again, this is critical.

Given:

users

1 Aarav

2 Emma

3 Liam

and:

documents

101 → user 1

102 → user 1

103 → user 2

Query:

```
SELECT
u.id,
COUNT(*) AS count_star,
COUNT(d.id) AS count_document_id
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
GROUP BY u.id;
```

For Liam:

COUNT(*) → 1

COUNT(d.id) → 0

Why?

Because the LEFT JOIN still produces one result row:

3 | Liam | NULL

Therefore:

COUNT(*)

counts the row.

But:

COUNT(d.id)

counts only **non-NULL d.id values**.

This is why, when counting child records after a LEFT JOIN, you will often use:

COUNT(d.id)

rather than:

COUNT(*)

15. Production Example — RAG

Imagine:

users

documents

document_chunks

embeddings

You want:

Every user and number of successfully embedded chunks.

A dangerous query:

```
SELECT
u.id,
COUNT(*)
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
LEFT JOIN document_chunks c
ON d.id = c.document_id
LEFT JOIN embeddings e
ON c.id = e.chunk_id
WHERE e.status = 'success'
GROUP BY u.id;
```

Problems?

There are potentially **multiple**.

You need to reason about:

1. LEFT JOIN
2. WHERE
3. NULL-extended rows
4. row multiplication
5. aggregation

A production-minded solution might instead use a pre-aggregated CTE:

```
WITH successful_embeddings AS (
SELECT
c.document_id,
COUNT(*) AS successful_embedding_count
FROM document_chunks c
JOIN embeddings e
ON c.id = e.chunk_id
WHERE e.status = 'success'
GROUP BY c.document_id
)
SELECT
u.id,
```

```

u.name,
COALESCE(SUM(se.successful_embedding_count), 0)
AS successful_embeddings
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
LEFT JOIN successful_embeddings se
ON d.id = se.document_id
GROUP BY
u.id,
u.name;

```

Notice the design:

embeddings

↓

pre-aggregate

↓

document-level metrics

↓

LEFT JOIN

↓

users

This is the kind of reasoning I want you to develop as a Data Engineer.

16. PostgreSQL CREATE TABLE

Now let's create a realistic table.

```

CREATE TABLE users (
id BIGINT,
name TEXT,
email TEXT,
country TEXT,
account_status TEXT,
created_at TIMESTAMPTZ
);

```

Then:

```

INSERT INTO users (
id,
name,
email,

```

```
country,  
account_status,  
created_at  
)  
VALUES  
(  
1,  
'Aarav',  
'aarav@example.com',  
'India',  
'active',  
NOW()  
);  
  
Check:  
SELECT *  
FROM users;
```

17. PostgreSQL Schemas

You can create:

```
CREATE SCHEMA analytics;
```

Then:

```
CREATE TABLE analytics.user_metrics (  
user_id BIGINT,  
document_count INTEGER,  
total_pages BIGINT  
);
```

Query:

```
SELECT *  
FROM analytics.user_metrics;
```

This is useful for separating:

public

application tables

analytics

analytical tables

staging

raw/staging tables

A production data platform may have:

raw

↓

staging

↓

core

↓

analytics

18. PostgreSQL Information Schema

PostgreSQL exposes metadata about database objects.

For example:

```
SELECT
table_name
FROM information_schema.tables
WHERE table_schema = 'public';
```

This is extremely useful for Data Engineering scripts.

You can inspect columns:

```
SELECT
column_name,
data_type
FROM information_schema.columns
WHERE table_name = 'documents';
```

19. PostgreSQL System Catalogs

PostgreSQL also exposes internal metadata through system catalogs such as:

pg_tables

pg_class

pg_attribute

pg_indexes

You'll encounter these later when doing database administration and optimization.

20. DAY 15 — Difficult Practice

Now **you solve these**.

Don't ask for answers before attempting.

Challenge 1 — PostgreSQL table design

Design a PostgreSQL table:

document_chunks

It should contain at least:

id

document_id

chunk_number

content

metadata

created_at

Choose appropriate PostgreSQL data types.

Write the complete:

CREATE TABLE

statement.

Challenge 2 — JSONB

Assume:

documents.metadata JSONB

Example:

```
{  
  "language": "en",  
  "source": "pdf",  
  "department": "engineering",  
  "pages": 50  
}
```

Write SQL to return documents where:

language = 'en'

Challenge 3 — LEFT JOIN mastery

Given:

users

documents

Write a query that returns:

user_id

user_name

processed_document_count

Requirements:

- return **every user**
- users with zero processed documents must remain
- count only documents where status = 'processed'
- don't count NULL rows
- don't accidentally turn the LEFT JOIN into an INNER JOIN

This is your **primary Day 15 challenge**.

Challenge 4 — Debugging

Consider:

```
SELECT
  u.id,
  u.name,
  COUNT(d.id) AS processed_documents
  FROM users u
  LEFT JOIN documents d
    ON u.id = d.user_id
  WHERE d.status = 'processed'
  GROUP BY
    u.id,
    u.name;
```

Answer these before rewriting it:

- What happens to users with no documents?
 - Why?
 - Does the COUNT(d.id) itself cause the problem?
 - Where should the child-table filter go if we want to preserve every user?
-

Challenge 5 — Production reasoning

Suppose your database contains:

10 million users

100 million documents

You need:

Every user + number of processed documents.

Compare these approaches conceptually:

Approach A

```
LEFT JOIN documents
  WHERE documents.status = 'processed'
```

Approach B

```
LEFT JOIN documents
  ON ...
```

AND documents.status = 'processed'

Approach C

Pre-aggregate:

WITH processed_docs AS (...)

then:

LEFT JOIN processed_docs

Explain:

1. Difference in result semantics
2. Which preserves zero-document users
3. Why pre-aggregation can be useful
4. What additional database feature might become important at this scale

Don't worry about performance numbers yet—we'll study indexes properly on Day 17.

21. Interview Questions

Answer these as if you're in a Data Engineer interview.

Q1

What is the difference between **SQL** and **PostgreSQL**?

Q2

Explain:

Database

Schema

Table

Q3

When would you use JSONB instead of separate relational columns?

Q4

Explain exactly why:

LEFT JOIN ...

WHERE right_table.condition

can behave like an INNER JOIN.

Q5

Why can:

COUNT(*)

return 1 while:

COUNT(child.id)

returns 0 after a LEFT JOIN?

Explanation Challenge

Explain this query without simply saying "it counts processed documents":

```
SELECT
u.id,
u.name,
COUNT(d.id) AS processed_documents
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
AND d.status = 'processed'
GROUP BY
u.id,
u.name;
```

Explain what happens for a user who has:

3 processed documents

2 failed documents

and then explain what happens for a user with:

0 documents

Day 15 Success Criteria

By the end of today, you should be able to reason confidently about:

PostgreSQL

↓

Database / Schema / Table

↓

PostgreSQL data types

↓

JSONB

↓

NULL semantics

↓

LEFT JOIN

↓

ON vs WHERE

↓

COUNT(*)

COUNT(column)

↓

Production data modeling

Most important target: I want your LEFT JOIN reasoning to become reliable across multiple variations, not just one memorized pattern.

Your task

Send me your answers for **Challenge 1–5** and the **5 interview questions**. I'll review them like an actual Data Engineering interview/code review and focus only on what you get wrong.